kmeans-mood

October 25, 2025

1 K-Means In Real Life: Clustering Habits of Positive Mood

1.1 Introduction

K-Means is an unsupervised method for identifying clusters within data sets, where K represents the number of clusters to be created. This method is helpful when you do not clearly understand the similarities of records in the data. By applying K-Means, these clusters can be identified and further researched.

1.2 Literature Review

In her article, Carolina Bento uses a data set related to workouts to explain how K-Means clusters can be identified as helpful. Using the workouts data set, Bento attempts to answer the question, "Which workouts are most similar?" (Bento, 2018). In his article, Zach Bobbitt walks through performing K-means clustering in Python using data from basketball players' stats (Bobbitt, 2022). Bobbitt clusters groups of players with similar points, assists, and rebound statistics in the player's data set. Both articles demonstrate the value of using K-means analysis on data sets to form similar data clusters. 2022). Bobbitt clusters groups of players with similar points, assists, and rebound statistics in the player's data set. Both articles demonstrate the value of using K-means analysis on data sets to form similar data clusters.

1.3 Research

1.3.1 Clustering Habits of Positive Daily Sentiment

Many people will have different habits and experiences in a given day. For example, the time a person wakes in the morning, the amount of sleep they got the night before, general health conditions, food or water intake, exercise levels, and other factors could influence the person's sentiment of the day. We could use K-means analysis to understand the clusters of habits that result in a person having a more positive sentiment of their day. Given sufficient data, this analysis could further delineate between habits that align with positive mood by race and gender and other factors (e.g., chronic health conditions, occupation, age, etc). In the experiment, we could ask users to track habits that impact mood and submit these data daily. Alternatively, we could leverage data the user shares via various providers (e.g., Apple et al.). As a starting point, we should define a set of habits that users can track, allowing us to analyze a consistent set of habits. In addition, we could allow users to create new habits to be tracked. Throughout the analysis, we must ensure sufficient data for each habit and its relationship to the user's daily sentiment.

1.3.2 Sample Dataset and Analysis

To better understand K-means implementation using Python, I created a sample data set and used the libraries specified in Bobbit's article to analyze the data. In Table 1, we can see a subset of sample habits collected by a research participant.

Table 1: Subset of Habit Tracker Data Set | Date | ExerciseDuration (minutes) | SleepDurationPriorNight (hours) | NumberOfMeetings (integer) | PerceivedEndOfDayMood | | — | — | — | — | — | | 2019-11-17 | 25 | 5 | 8 | POOR | | 2019-11-18 | 30 | 6 | 6 | POOR | | 2019-11-19 | 45 | 8 | 1 | GOOD | | 2019-11-21 | 30 | 6 | 5 | POOR | | 2019-11-22 | 45 | 7 | 2 | POOR |

Through analysis of the data using pair plots in Seaborn, I determined that the habits "Number of Meetings," "Duration of Sleep Prior Night (hours)," and "Duration of Exercise (minutes)" correlate with the daily sentiment of this participant. Initial analysis of the data shown in Figure 1 (See Appendix: Codebook) demonstrates that a good K value for this data set is 3.

Figure 2 (See Appendix: Codebook) shows the relationship between the habits and sentiments and the 3 clusters formed by K-means analysis. Because K-means analysis in Python requires numeric values, we have factionalized the PerceivedEndOfDayMood variable where Bad is represented by the number zero and Good is represented by one. Neutral is represented by the number two. From the analysis, we see that cluster zero data contains days where the user has had a good night's sleep, sufficient exercise, and a low number of meetings for the day. A high number of meetings and low levels of sleep and exercise form cluster two.

1.4 Conclusion

Using unsupervised methods, k-means analysis provides a way to identify similar groups within a data set. This analysis assists with forming clusters that can be further explored to answer research questions. In our research example, we demonstrate that with a set of habit-tracking data, we can identify similarities in groups that lead to positive sentiment about a person's day. By expanding the population of participants and trackable habits, we could find correlations in habits that lead to generally positive sentiments.

1.5 Appendix: References

- Bento, C. (2018, December 3). K-Means in real life: Clustering workout sessions - towards data science. Medium. Accessed 04/20/2024. https://towardsdatascience.com/k-means-in-real-life-clustering-workout-sessions-119946f9e8dd
- Cheng, J. (2021, December 6). Data Exploration and Visualization with Seaborn Pair Plots. Medium. Accessed 04/20/2024. https://medium.com/@jaimejcheng/data-exploration-and-visualization-with-seaborn-pair-plots-40e6d3450f6d
- Bobbitt, Z. (2022, August 31). K-Means Clustering in Python: Step-by-Step Example. Statology. Accessed 4/20/2024. https://www.statology.org/k-means-clustering-in-python/

1.6 Appendix: Code Book

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

1.6.1 Load and review the dataset

[2]:
```
# Create Dataframe
df = pd.read_csv('../data/habits_mood.csv')
```

[3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 33 entries, 0 to 32
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Date                     33 non-null     object
 1   DaysBetweenTracking      33 non-null     int64
 2   WaterInakeMl             33 non-null     int64
 3   ExerciseDuration         33 non-null     int64
 4   PrimaryTypeOfExercise    33 non-null     object
 5   SleepDurationPriorNight  33 non-null     int64
 6   NumberOfMeetings         33 non-null     int64
 7   DurationOfMeetings       33 non-null     int64
 8   UnplannedWorkInjected    33 non-null     float64
 9   MinutesSpentOnBreak      33 non-null     int64
 10  PerceivedEndOfDayMood    33 non-null     object
dtypes: float64(1), int64(7), object(3)
memory usage: 3.0+ KB
```

[4]:
```
df.sample(5, random_state=42)
```

[4]:

	Date	DaysBetweenTracking	WaterInakeMl	ExerciseDuration	PrimaryTypeOfExercise	SleepDurationPriorNight	NumberOfMeetings	DurationOfMeetings	UnplannedWorkInjected	MinutesSpentOnBreak	PerceivedEndOfDayMood
31	1/9/2020	5	39	40	CARDIO	4	3	120	2.0	55	Good
15	12/10/2019	2	42	45	WEIGHT	6	3	120	2.0	55	Good
26	12/30/2019	1	33	60	WEIGHT	8	2	60	0.0	30	Great
17	12/14/2019	3	28	40	CARDIO	5	2	60	0.0	30	Great
8	12/1/2019	1	36	35	WEIGHT	7	4	120	2.0	55	Good

```
26                    90                    1.5                   60
17                    90                    1.5                   50
8                    210                    3.5                   45

   PerceivedEndOfDayMood
31                  POOR
15                  POOR
26                  GOOD
17                  POOR
8                NEUTRAL
```

1.6.2 Feature Engineering

[5]:
```
df['PrimaryTypeOfExercise'] = pd.factorize(df['PrimaryTypeOfExercise'])[0]
```

[6]:
```
df['PerceivedEndOfDayMood'] = df['PerceivedEndOfDayMood'].map({'GOOD': 1,␣
↪'POOR': 0, 'NEUTRAL': 2})
```

[7]:
```
df['Date'] = pd.to_datetime(df['Date'])
df['Date'] = df['Date'].values.astype(np.int64) // 10**9
```

1.6.3 Limit features for clustering

[8]:
```
df =␣
↪df[['Date','ExerciseDuration','SleepDurationPriorNight','NumberOfMeetings','PerceivedEndOfD
```

1.6.4 Drop rows with NA values

[9]:
```
# drop rows with NA values in any columns
df = df.dropna()
```

1.6.5 Scale the dataframe

[10]:
```
# create scaled Dataframe where each variable has mean of 0 and standard␣
↪deviation of 1
scaled_df = StandardScaler().fit_transform(df)
```

[11]:
```
# view first five rows of scaled dataframe
print(scaled_df[:5])
```

```
[[-1.62437506 -1.75030306 -1.30403763  2.53533744 -0.8977584 ]
 [-1.56218901 -1.31601734 -0.6209703   1.55102996 -0.8977584 ]
 [-1.50000296 -0.01316017  0.74516436 -0.90973873  0.748132  ]
 [-1.37563085 -1.31601734 -0.6209703   1.05887623 -0.8977584 ]
 [-1.3134448  -0.01316017  0.06209703 -0.41758499 -0.8977584 ]]
```

1.6.6 Identify Number of Clusters to create

[12]:
```
# initialize kmeans parameter
kmeans_kwargs = {
    "init": "random",
    "n_init": 10,
    "random_state": 1,
}
```

[13]:
```
# create list to hold SSE values for each K
# SSE = Sum of Squared Errors
sse = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, **kmeans_kwargs)
    kmeans.fit(scaled_df)
    sse.append(kmeans.inertia_)
```

[20]:
```
# visualize results
plt.plot(range(1, 11), sse)
plt.title("Figure 1: Sum of Squared Errors Analysis")
plt.xticks(range(1, 11))
plt.xlabel("Number of Clusters K")
plt.ylabel("SSE")
plt.show()
```

1.6.7 Instantiate K-Means Clusters

[15]:
```
# instantiated the k-means class, using optimal number of clusters
kmeans = KMeans(init="random", n_clusters=3, n_init=11, random_state=1)
```

[16]:
```
# fit k-means algorithm to data
kmeans.fit(scaled_df)
```

[16]:
```
KMeans(init='random', n_clusters=3, n_init=11, random_state=1)
```

[17]:
```
# view cluster assignments for each observation
df['cluster'] = kmeans.labels_
```

[18]:
```
display(df)
```

	Date	ExerciseDuration	SleepDurationPriorNight	NumberOfMeetings	\
0	1573948800	25	5	8	
1	1574035200	30	6	6	
2	1574121600	45	8	1	
3	1574294400	30	6	5	
4	1574380800	45	7	2	

5	1574640000	25	5	6
6	1574726400	35	7	5
7	1575072000	25	5	5
8	1575158400	35	7	4
9	1575244800	45	8	1
10	1575331200	50	10	1
11	1575417600	45	8	1
12	1575590400	30	6	8
13	1575676800	45	7	3
14	1575763200	50	9	1
15	1575936000	45	6	3
16	1576022400	60	8	3
17	1576281600	40	5	2
18	1576627200	35	4	5
19	1576713600	45	7	2
20	1576800000	50	8	2
21	1576972800	60	7	3
22	1577059200	60	8	2
23	1577404800	40	5	2
24	1577491200	60	7	1
25	1577577600	60	8	1
26	1577664000	60	8	2
27	1577750400	50	9	1
28	1577836800	60	8	2
29	1578009600	45	6	1
30	1578096000	60	8	1
31	1578528000	40	4	3
32	1578614400	60	8	1

	PerceivedEndOfDayMood	cluster
0	0	2
1	0	2
2	1	0
3	0	2
4	0	1
5	0	2
6	2	0
7	0	2
8	2	0
9	1	0
10	1	0
11	1	0
12	0	2
13	0	1
14	1	0
15	0	1
16	1	0
17	0	1

18	0	2
19	0	1
20	1	0
21	0	1
22	1	0
23	0	1
24	0	1
25	1	0
26	1	0
27	1	0
28	1	0
29	0	1
30	1	0
31	0	1
32	1	0

1.6.8 Visualize K-Means Clusters

[27]:
```
# visualize pair plot
sns.set_style("darkgrid")
sns.set_palette("dark")
sns.pairplot(df,
             hue='cluster',
             markers=['v','^','o'],
             palette = ['red','blue','green'],
             vars=['ExerciseDuration',
                   'SleepDurationPriorNight',
                   'NumberOfMeetings',
                   'PerceivedEndOfDayMood']).fig.suptitle("Figure 2: Habit␣
 ↪Correlation to Sentiment", y=1.02)
plt.show()
```

Figure 2: Habit Correlation to Sentiment